Module 04

Dockerfile and Registry

DevOps end to end services & solutions

Agenda

- Dockerfile – Basic commands
- Dockerfile - Based on node:14 OS
- Docker Build and Docker Run Image
- Multi-Stage Build
- Docker Registry - Tag and Push Image
- Hands-On – Lab-04 – Dockerfile and Registry

What is a Dockerfile?

Blueprint

A script containing a set of instructions to automate the building of Docker images.

Setup

Defines everything needed to set up an app inside a container (OS, code, dependencies).

Automation

Contains all the commands a user could call on the command line to assemble an image.

Dockerfile – Basic Commands

- **FROM:** Specifies the base image.
- **WORKDIR:** Sets the working directory inside the container.
- **COPY & ADD:** Copies files from host to container.
- **RUN:** Executes commands during build time.
- **CMD & ENTRYPOINT:** Defines the default command when running the container.
- **EXPOSE:** Opens a port for the container.

```
FROM ubuntu:22.04
RUN apt-get update -y
RUN apt-get upgrade -y
RUN apt-get install -y curl
```

Dockerfile - Based on node:14 OS

```
# Use an official Node.js runtime as a parent image
FROM node:14
# Set the working directory in the container
WORKDIR /usr/src/app
# Copy package.json and package-lock.json to the working directory
COPY package*.json ./
# Install any needed dependencies
RUN npm install
# Copy the rest of the application code to the working directory
COPY . .
# Make the container's port 3000 available to the outside
EXPOSE 3000
# Define the command to run the application
CMD ["npm", "start"]
```

Docker Build and Run

Docker Build

Creates an image from a Dockerfile. Packages your code to be shipped anywhere.

```
docker build -t node-app .
```

Docker Run

Creates a running container from an image. Use -d for background mode.

```
docker run -p 3000:3000 node-app
# Or run in background
docker run -d -p 3000:3000 node-app
```

Multi-Stage Build

Multi-stage builds allow you to drastically reduce the size of your final image by using multiple FROM instructions.

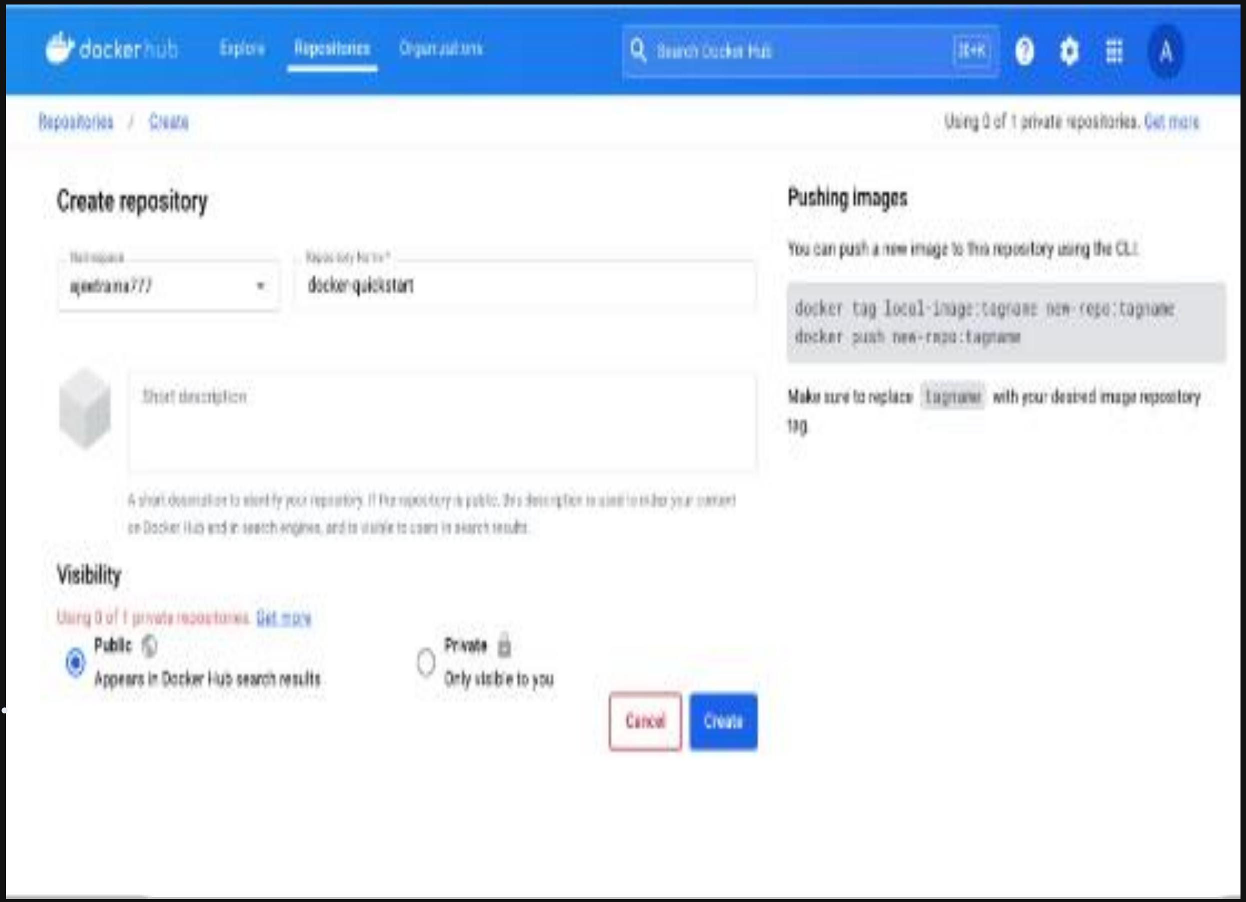
You can copy artifacts from one stage to another, leaving behind all the build tools and intermediate files.

```
# Stage - Development
FROM Dev-Image as dev
...          stage 0
...

# Stage - Testing
FROM Test-Image as test
COPY --from=dev /src/app .
...          stage 1
...

# Stage - Production
FROM Minimum-Image as prod
COPY --from=dev /src/app .
...          stage 2
...
```

Docker Registry – Tag and Push

Steps to Publish

- **Login:** Authenticate with Docker Hub.
- **Tag:** Rename your local image to match your registry repository.
- **Push:** Upload the image to the registry.



Tag & Push Commands

```
# 1. Login to Docker Hub
docker login
# 2. Tag the image
# docker tag /:
docker tag myapp mydockeruser/myapp:latest
# 3. Push the image
docker push mydockeruser/myapp:latest
# 4. Pull (Verify)
docker pull mydockeruser/myapp:latest
```

Hands-On – Lab 04: Dinner Suggestion

- Create a app.py application (Python Flask).
- Wrap it with a Dockerfile.
- Build the image.
- Run the image and access via browser.
- Test via interactive mode.
- Tag and Push to Docker Hub.

Questions?

Thank you for participating.

Feel free to reach out:
https://dinghy-e2e.com